

Lab Assessment Rubrics of Digital System Design (CPE-344)

Lab Number	01
Lab Title	Introduction to Xilinx software
Submitted by	
Names	Registration Number
Sumaira Safeer	FA22-BCE-019

Rubrics name & number		Marks	
		In-Lab	Post-Lab
Engineering Knowledge	R2: Use of Engineering Knowledge and follow Experiment Procedures: Ability to follow experimental procedures, control variables, and record procedural steps on lab report.		
	R3: Interpretation of Subject Knowledge: Ability to interpret and explain mathematical and/or visual forms, including equations, diagrams, graphics, figures, and tables.		
Problem Analysis	R6: Experimental Data Analysis: Ability to interpret findings, compare them to values in the literature, identify weaknesses and limitations.		
Design	R7: Implementing Design Strategy: Ability to execute a solution taking into consideration design requirements and pertinent contextual elements. [Block Diagram/Flow chart/Circuit Diagram]		
	R8: Best Coding Standards: Ability to follow the coding standards and programming practices.		
Modern Tools Usage	R9: Understand Tools: Ability to describe and explain the principles behind and applicability of engineering tools.		
	R11: Tools Evaluation: Ability to simulate the experiment and them using hardware tools to verify the results.		
Individual and Teamwork	R12: Individual Work Contributions: Ability to carry out individual responsibilities.		
	R13: Management of Teamwork: Ability to appreciate, understand and work with multidisciplinary team members.		

Rubrics to follow

Rubrics	R2	R3	R5	R6	R7	R8	R9	R11	R12	R13
In-Lab										
Post-Lab										

Lab No: 01

Title:

Introduction to Xilinx Software, Verilog modeling and implementation of different logic gates, half adder and full adder on Xilinx.

Objective:

The main objective of this lab is:

- To get familiar with Xilinx software.
- To learn the procedure to implement digital circuits using Verilog.
- To verify the truth tables of different logic gates using simulation.

Task #01:

Implementation of AND logic gates in a single module using Xilinx.

Verilog Code:

```
module andGate(c, a, b);  
    input a,b;  
    output c;  
    and(c,a,b); // and function where c is the output, a, b input  
endmodule
```

Verilog Test Bench:

```
module andGateTest;  
    reg a; //input  
    reg b;  
    wire c; // Outputs  
  
    // Instantiate the Unit Under Test (UUT)  
    andGate uut (  
        .c(c),  
        .a(a),  
        .b(b)  
    );  
  
    initial begin  
        a = 0; b = 0; #100;  
        a = 0; b = 1; #100;  
        a = 1; b = 0; #100;  
        a = 1; b = 1;  
    end  
endmodule
```

Waveform:

Following waveforms are verifying our truth table for the AND GATE logic.

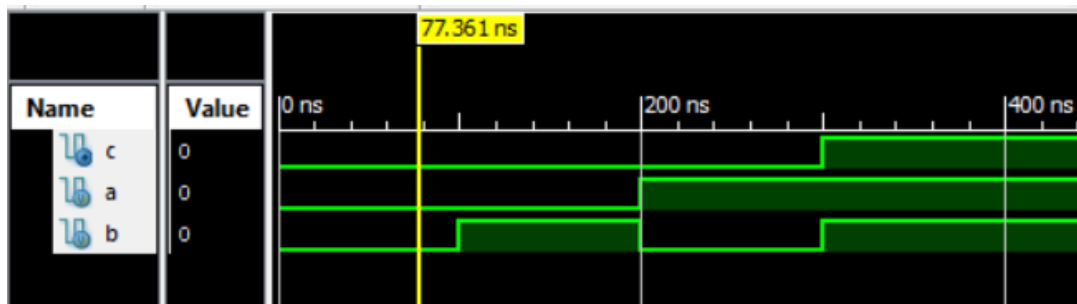


Figure 1: When both inputs are zero(0), our output is also zero(0)

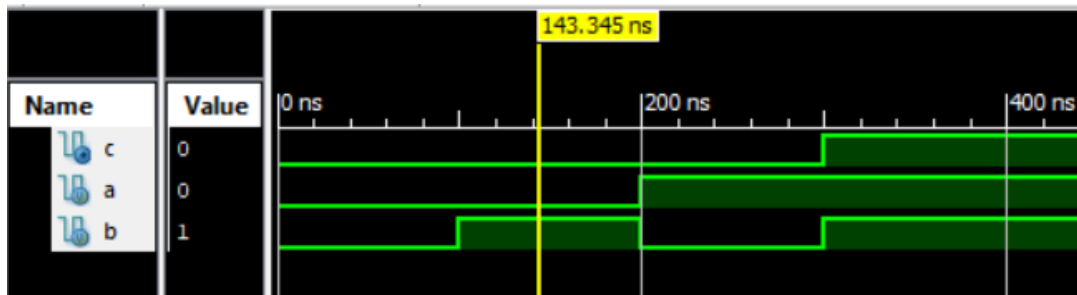


Figure 2: When our first input is zero (0) but second is one (1), but still our output remains zero (0)

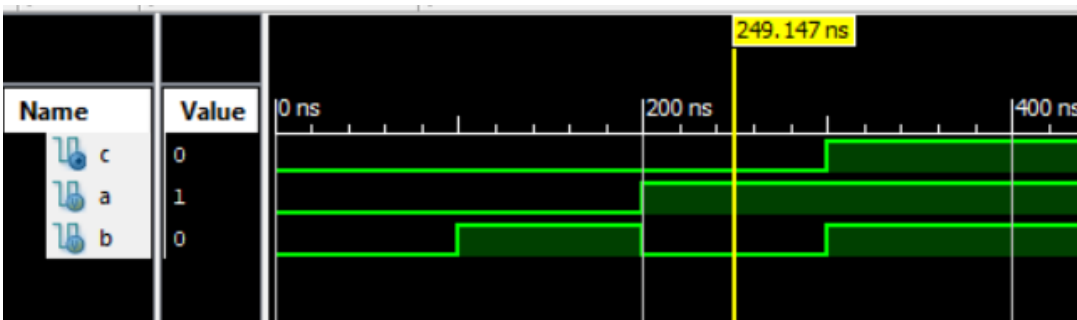


Figure 3: When our first input is one (1) but second is zero (0), but still output remains zero (0)

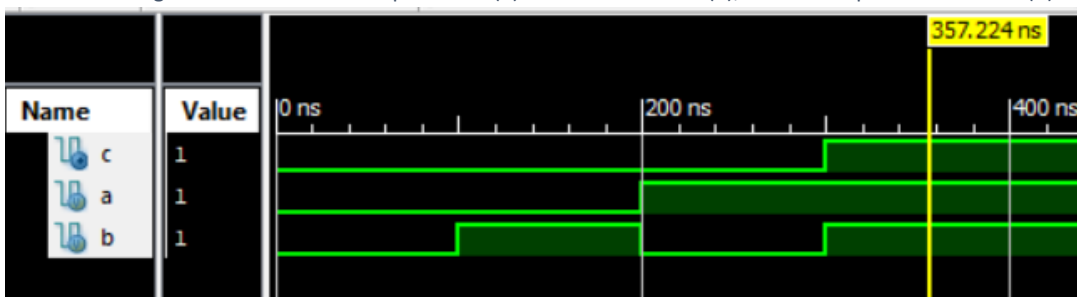


Figure 4 : When both inputs are one (1), output will be one (1)

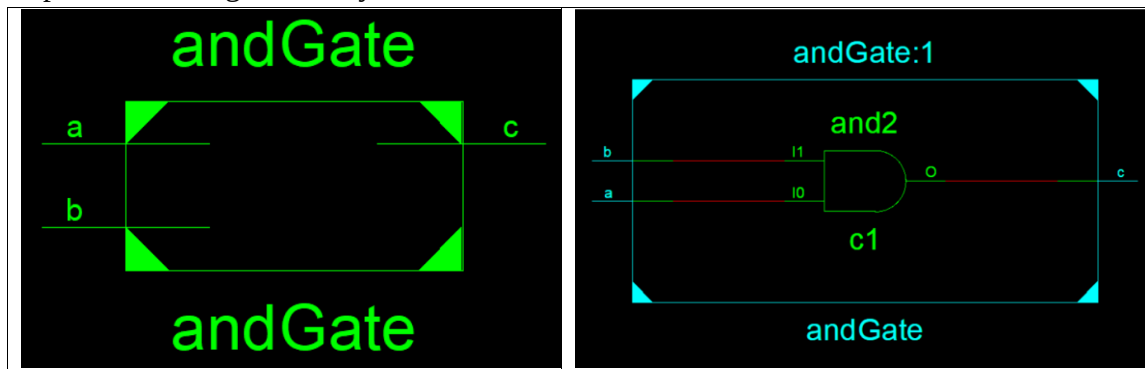
Wave Description (Based on Input Combinations for AND Gate):

- **Case 1: $a = 0, b = 0 \rightarrow c = 0$**
Output remains low when both inputs are low.
- **Case 2: $a = 0, b = 1 \rightarrow c = 0$**
Even if one input is high, the output stays low.

- **Case 3: $a = 1, b = 0 \rightarrow c = 0$**
Output remains low because one input is zero.
- **Case 4: $a = 1, b = 1 \rightarrow c = 1$**
Only when both inputs are high, the output goes high.

RTL Diagram:

The RTL diagram generated by Xilinx shows a direct **AND gate symbol** with two inputs and one output, confirming correct synthesis.



Task #02:

Design a Half adder using either NAND gate or NOR gate. Verify waveform of above gates by writing test module.

Truth Table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Mathematical Modeling

The half adder performs binary addition of two single bits. The equations are:

Sum Output:

- $\text{Sum} = A \oplus B$ (XOR operation)
- Using NAND gates: $\text{Sum} = ((A \cdot B))' \cdot ((A') \cdot (B'))'$

Carry Output:

- $\text{Carry} = A \cdot B$ (AND operation)
- Using NAND gates: $\text{Carry} = ((A \cdot B))'$

Verilog Code:

```
module halfAdderNAND(S, C, A, B);
    input A, B;
```

```

output S, C;
wire w1, w2, w3; // Internal wires for connections
// Implement Half Adder using only NAND gates
nand nand1 (w1, A, B);
nand nand2 (w2, A, w1);
nand nand3 (w3, w1, B);
nand nand4 (S, w2, w3);
nand nand5 (C, w1, w1); // This acts as a NOT gate for w1
endmodule

```

Verilog Test Bench:

```

module halfAdderNANDTest;
    reg A, B; // Inputs
    wire S, C; // Outputs

    // Instantiate the Unit Under Test (UUT)
    halfAdderNAND uut (
        .S(S),
        .C(C),
        .A(A),
        .B(B)
    );
    initial begin
        // Initialize Inputs
        A = 0;
        B = 0;
        #100; // Wait 100 time units

        // Apply all possible input combinations
        A = 0; B = 1;
        #100;
        A = 1; B = 0;
        #100;
        A = 1; B = 1;
        #100;
        $finish; // End simulation
    end
endmodule

```

Waveform

The simulation waveform displays the following signals over time:



Figure 5: When our first and second input is zero (0).



Figure 6: When our first input is zero (0) but second is one (1).

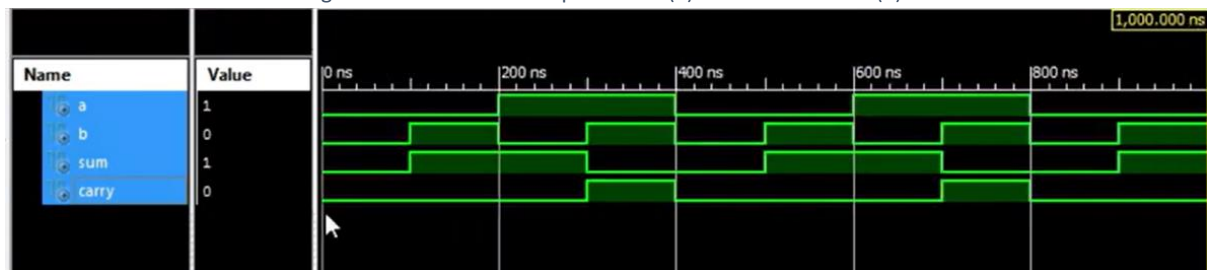


Figure 7: When our first input is one (1) but second is zero (0).



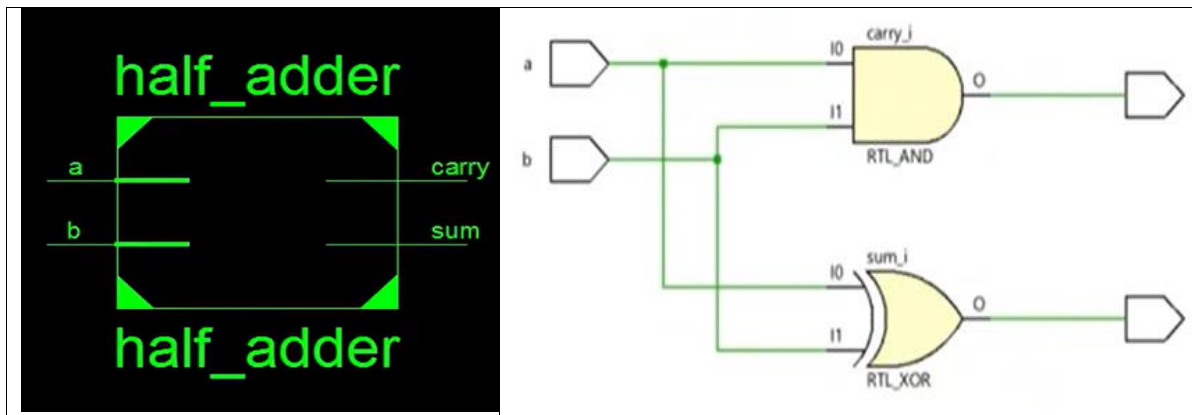
Figure 8: When our first input is one (1) but second is one (1).

Wave Description (Based on Input Combinations)

- When inputs are (0,0) → Sum=0, Carry=0
- When inputs are (0,1) → Sum=1, Carry=0
- When inputs are (1,0) → Sum=1, Carry=0
- When inputs are (1,1) → Sum=0, Carry=1

RTL Schematic

- The RTL schematic shows the structural implementation of the half adder using NAND gates.



Conclusion:

In this lab, successfully implemented and simulated basic digital circuits using Xilinx software and Verilog HDL. The **AND gate** was tested, and its output was verified to be high only when both inputs were high, confirming its truth table. The **Half Adder** was designed and simulated, showing that the **Sum** output followed XOR logic while the **Carry** output followed AND logic, exactly as expected. Finally, the **Full Adder** was implemented to extend binary addition with a carry-in input, and the simulation results matched the theoretical truth table for all eight input combinations.